

Git

1.什么是git:

git是一个免费开源的分布式版本控制系统，使用repository来管理文件

版本控制系统：

- 集中式：SVN,CVS
- 分布式：Git, Mercurial

2.git的使用方式：

- 命令行
- 图形化界面（GUI）
- IDE插件/扩展

3.版本信息：

```
git -v
```

4.注册用户名和邮箱以及保存和显示

```
git config --global user.name "name"
git config --global user.email email@
git config --global credential.helper store
git config --global --list
```

5.新建仓库

- 在本地新建一个仓库

```
git init
```

- 从远程服务器origin中克隆

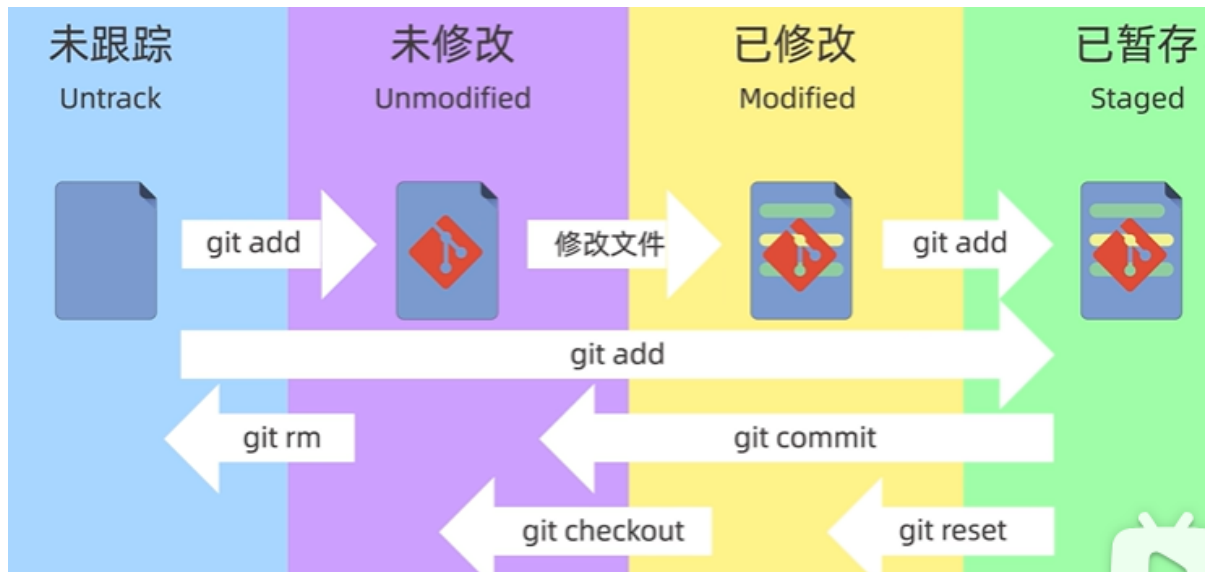
```
git clone 仓库地址
```

6.git的工作区域和文件状态：

- 工作区Working Directory:工作目录，文件目录
- 暂存区Staging Area/Index：保存即将提交到仓库的内容
- 本地仓库Local Repository：就是git init创建的仓库



- 文件状态：

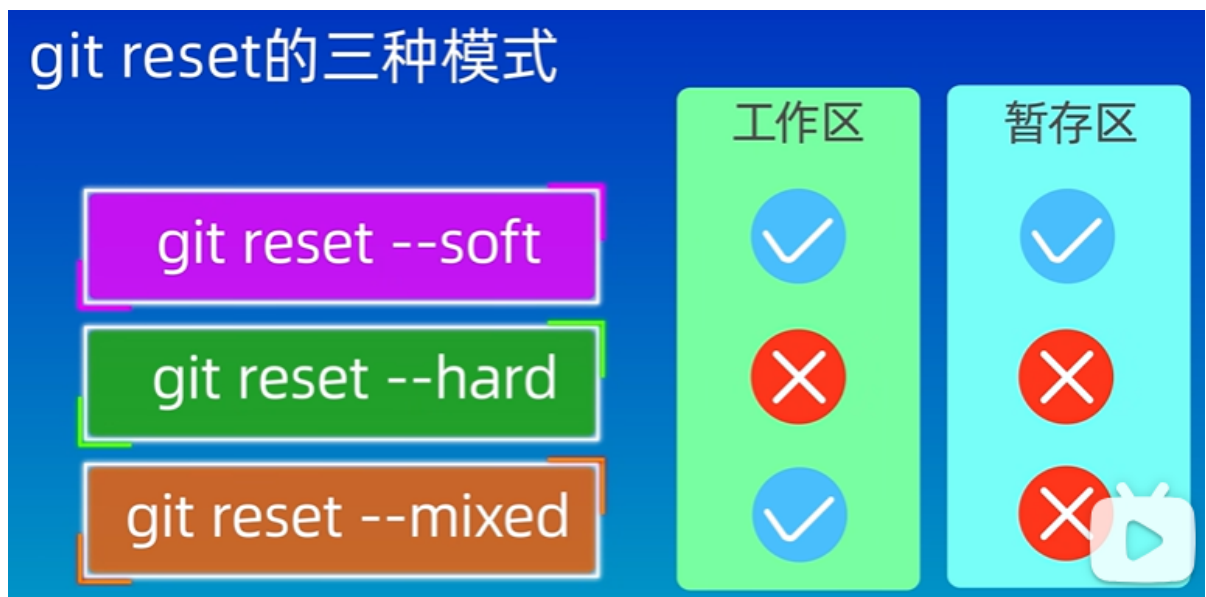


7.将文件添加到仓库中：

- `git init` 创建仓库
- `git status` 查看仓库的状态
- `git add` 添加到暂存区(可以配合通配符使用)
- `git commit` 提交
- `git log` 检查提交信息

8.回退版本：

- `git reset`:



9.git diff：查看文件在工作区、暂存区以及版本库之间的差异；查看不同版本之间的差异；或者文件在两个分支之间的差异

git diff	工作区	VS	暂存区	
git diff HEAD	工作区	+	暂存区	VS 本地仓库
git diff --cached / git diff --staged			暂存区	VS 本地仓库
git diff <commit_hash> <commit_hash> / git diff HEAD~ HEAD	比较提交之间的差异			
git diff <branch_name> <branch_name> /	比较分支之间的差异			

10.从版本库中删除文件:

git rm 文件

rm file; git add file	先从工作区删除文件, 然后再暂存删除内容
git rm <file>	把文件从工作区和暂存区同时删除
git rm --cached <file>	把文件从暂存区删除, 但保留在当前工作区中
git rm -r *	递归删除某个目录下的所有子目录和文件
	删除后不要忘记提交

11..gitignore文件: 让我们忽略掉一些不应该加入到版本库中的文件, 让仓库体积更小, 更干净

应该忽略的文件:

应该忽略哪些文件

.gitignore

- 忽略日志文件和文件夹 ✓
- 忽略所有.class文件 ✓
- 忽略所有.o文件 ✓
- 忽略所有.env文件 ✓
- 忽略所有.zip和tar文件 ✓
- 忽略所有.pem文件 ✓
-

- 系统或者软件自动生成的文件
- 编译产生的中间文件和结果文件
- 运行时生成日志文件、缓存文件、临时文件
- 涉及身份、密码、口令、秘钥等敏感信息文件

.gitignore文件的匹配规则:

从上到下逐行匹配, 每一行表示一个忽略模式。

空行或者以#开头的行会被git忽略, 一般空行用于可读性的分隔, #一般用于注释。

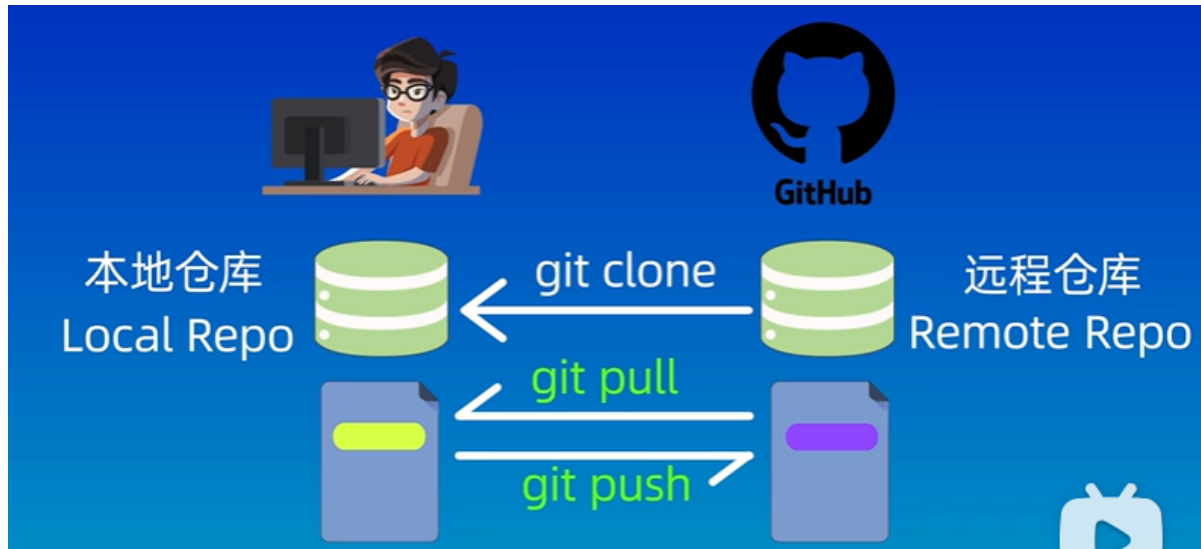
使用标准的Blob模式匹配 (类似正则表达式), 例如:

- *通配任意个字符
- ? 匹配单个字符
- []表示匹配列表中的单个字符，比如：[abc]表示a/b/c
- 两个**表示匹配任意的中间目录
- 中括号可以使用短中线连接
- ! 表示取反

12.远程仓库：remote repository

GitHub

SSH连接



git push 把本地仓库的修改推送给远程仓库

git pull 把远程仓库的修改拉取到本地仓库

13.把本地仓库传送到远程仓库：

总结

添加远程仓库：

Step 1

git remote add <远程仓库别名> <远程仓库地址>

Step 2

git push -u <远程仓库名> <分支名>

查看远程仓库：

git remote -v

拉取远程仓库内容：

git pull <远程仓库名> <远程分支名>:<本地分支名>

相同可省略冒号后面部分

14.GitLab和Gitee:代码托管平台

- Gitee:码云
- GitLab：极狐

15.在GUI和IDE中使用git:

- GitHub Desktop,SourceTree,GitKraken都可以作为图形化界面
- VSCode中使用Git

布局：菜单栏、侧边栏、编辑区、终端区
打开命令面板：Ctrl + Shift + ` / Command + Shift + `
配置文件：settings.json
打开终端：Ctrl + Shift + `
插件和扩展：在侧边栏中的扩展界面中搜索和安装
Git操作：在侧边栏中的源码管理器UI界面中操作

16.Git中的分支：

git branch 显示分支

git checkout 分支名：切换分支

git merge 合并分支

总结

查看分支列表：	\$ git branch
创建分支：	\$ git branch branch-name
切换分支：	\$ git checkout branch-name 【推荐】\$ git switch branch-name
合并分支：	\$ git merge branch-name
删除分支：	【已合并】\$ git branch -d branch-name 【未合并】\$ git branch -D branch-name

17.解决分支冲突：

两个分支未修改同一个文件的同一处位置：Git 自动合并

两个分支修改了同一个文件的同一处位置：产生冲突

解决方法：

Step1 - 手工修改冲突文件，合并冲突内容

Step2 - 添加暂存区 `$ git add file`

Step3 - 提交修改 `$ git commit -m "message"`

中止合并：当不想继续执行合并操作时可以使用下面的命令来中止合并过程：

`$ git merge --abort`

18.回退和rebase（变基）：

```
git rebase branch-name
```

可以按不同分支进行变基,结果都是一条直线,但是顺序不同.

19.分支管理和工作流模型:

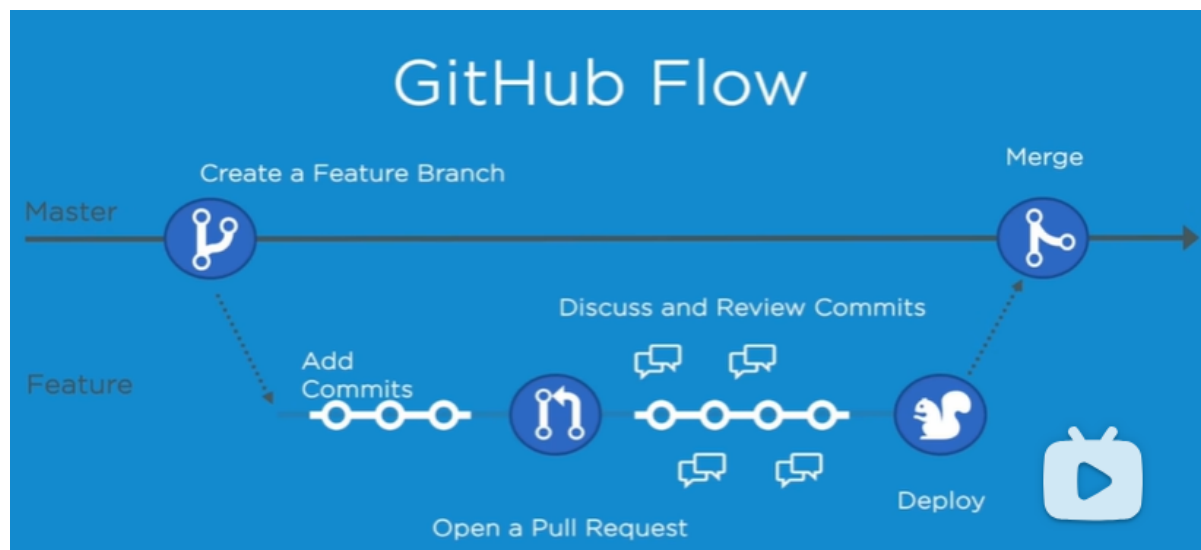
GitFlow模型:

- main:主线分支
- hotfix:问题修复分支
- release:预发布分支
- develop:开发分支
- feature-login-page/feature:功能分支

主要分支:main和develop

辅助分支:其他的分支

GitHubFlow模型:



分支命名

推荐使用带有意义的描述性名称来命名分支

版本发布分支/Tag示例： v1.0.0

功能分支示例： feature-login-page

修复分支示例： hotfix-#issueid-desc

分支管理

定期合并已经成功验证的分支, 及时删除已经合并的分支

保持合适的分支数量

为分支设置合适的管理权限